

PROJECT REPORT

BIS AI Assistant

An Intelligent Conversational AI for the Bureau of Indian Standards

1. Abstract

The Bureau of Indian Standards (BIS) serves as the National Standards Body of India, providing critical guidelines for product quality, consumer safety, and regulatory compliance. However, navigating the vast repository of BIS standards, Quality Control Orders (QCOs), and certification schemes can be challenging for average citizens and businesses due to technical jargon, language barriers, and the sheer volume of documents.

This project introduces the 'BIS AI Assistant,' an intelligent, multilingual, and voice-enabled conversational AI system. Built using a modern Retrieval-Augmented Generation (RAG) pipeline, the assistant accurately retrieves information exclusively from official BIS documents, ensuring zero hallucinations and high reliability. The system features a responsive React frontend, a high-performance FastAPI backend, a vector database for semantic search, and an integrated real-time data scraper portal to keep the knowledge base up-to-date.

2. Introduction

2.1 Problem Statement

Citizens and manufacturers require access to Indian Standards (IS), Conformity Assessment Schemes, and Hallmarking regulations. The current process involves manually searching through thousands of PDF documents and web pages. Key challenges include:

- Information Overload: Data is scattered across multiple portals.
- Language Barriers: Essential documents are often available only in English or complex technical language.
- Accessibility: Lack of intuitive search mechanisms for users unfamiliar with standard numbering.

2.2 Proposed Solution

The BIS AI Assistant bridges this gap by acting as a 24/7 intelligent guide. By leveraging Large Language Models (LLMs) and Vector Embeddings, the system understands natural language queries (even if colloquial or misspelled), retrieves the exact official paragraphs, and formulates a precise, easy-to-understand answer with citations.

3. System Architecture

The architecture is designed for scalability, low latency, and high accuracy. It follows a decoupled client-server model.

3.1 Frontend (User Interface)

- Framework: React.js powered by Vite for rapid development and optimized builds.
- Styling: Tailwind CSS / Custom CSS for a modern, responsive, and accessible interface.

- Features: Real-time chat interface with markdown rendering, Voice-to-Text and Text-to-Speech integration, Multilingual support, and a dedicated Scraper Portal for administrators.

3.2 Backend (API & Business Logic)

- Framework: FastAPI (Python) for asynchronous, high-throughput request handling.
- Database: PostgreSQL (via Supabase) for robust session management and data persistence.
- Vector Store: Supabase pgvector for storing and retrieving high-dimensional document embeddings.
- LLM Orchestration: Custom RAG pipeline capable of routing requests to OpenRouter or falling back to local Ollama instances.

4. Technical Implementation Details

4.1 Retrieval-Augmented Generation (RAG) Pipeline

The core of the assistant is the RAG engine, which operates in three phases:

- Ingestion & Chunking: Official BIS PDFs and HTML pages are parsed using PyMuPDF and BeautifulSoup. The text is split into chunks of 1000 characters using RecursiveCharacterTextSplitter.
- Embedding: Each chunk is passed through an Embedding Model (e.g., OpenAI text-embedding-3-small) to convert semantic meaning into mathematical vectors. These vectors are stored in Supabase.
- Retrieval & Generation: When a user asks a question, the query is converted into a vector. The system performs a cosine similarity search to find the top 5 most relevant document chunks. These chunks are appended to the system prompt, forcing the LLM to generate answers **only** using the provided context.

4.2 Query Normalization & Intent Recognition

Before searching the vector database, the system intercepts the user's query and performs intent normalization. For example, if a user asks about 'water,' the system expands the search terms to include 'IS 10500, IS 14543, packaged drinking water.' This drastically improves retrieval accuracy for non-technical users.

4.3 Multi-Tier LLM Fallback Mechanism

To ensure the system never goes down during high traffic or API outages, it implements a multi-tier fallback:

- Primary: Custom API endpoints.
- Secondary: OpenRouter API utilizing highly capable models like google/gemma-4-31b-it.
- Tertiary: Local Ollama instances running local models to guarantee privacy.

5. Security & Deployment

The application has been architected to be completely serverless and cloud-native:

- Vercel: Hosts the Vite frontend as static assets and runs the FastAPI backend as Serverless Functions via

the `api/index.py` endpoint.

- Supabase: Replaces local SQLite and ChromaDB, providing enterprise-grade PostgreSQL. The `match_documents` RPC function ensures vector searches are computed directly on the database server.

6. Future Scope

- WhatsApp Bot Integration: Extending the AI assistant to WhatsApp via the Meta Graph API to reach citizens who do not use web browsers.
- Image Recognition: Allowing users to upload pictures of ISI marks or HUID codes to instantly verify authenticity using Vision LLMs.

7. Conclusion

The BIS AI Assistant successfully demonstrates how modern Generative AI and RAG architectures can be utilized to democratize access to national standards. By transforming static, difficult-to-read PDFs into an interactive, multilingual, and voice-enabled conversational agent, this project significantly enhances the accessibility of the Bureau of Indian Standards for all citizens.